Contents

Ride-Hailing Platform — HLD & LLD (SDE2 Assignment)

Stack: Node.js + Express + TypeScript · PostgreSQL via Prisma · Redis (geo search, driver state, ride-offer state) · BullMQ (queues/workers) · Socket.IO (real-time)

HLD Explanation

Overview

The system is a single Express/TypeScript API (`src/app.ts`, `src/server.ts`) consumed by two clients — the Rider App and the Driver App. The API is intentionally **stateless**:

every request is self-contained and can be served by any running instance. Durable state lives in **Postgres** (via Prisma — models `Driver`, `Ride`, `Payment`, `TripLocation`); fast-changing, high-frequency state lives in **Redis** (`drivers:geo`, `driver:${driverId}`, ride-offer records). Multi-step work that doesn't need to block the HTTP response — matching, notifications, payments — is handed off to **BullMQ** workers, and live updates are pushed to clients through **Socket.IO** rooms.

Main Components

Component	Responsibility
Rider App / Driver App	REST calls for actions; WebSocket for live updates
API Server (Express)	`src/routes/*` → `src/controllers/*` → `src/services/*`; validates input, talks to Postgres/Redis, enqueues jobs
Postgres (Prisma)	Durable source of truth: `Driver`, `Ride`, `Payment`, `TripLocation`
Redis	`drivers:geo` (geo index), `driver:${driverId}` (live status), ride-offer records (bid tokens)
BullMQ Queues/Workers	`ride-matching`, `notification`, `payment-processing` queues, each with a dedicated worker
Socket.IO	Rooms `driver:${driverId}` and `ride:${rideId}` for targeted real-time push

Data Flow

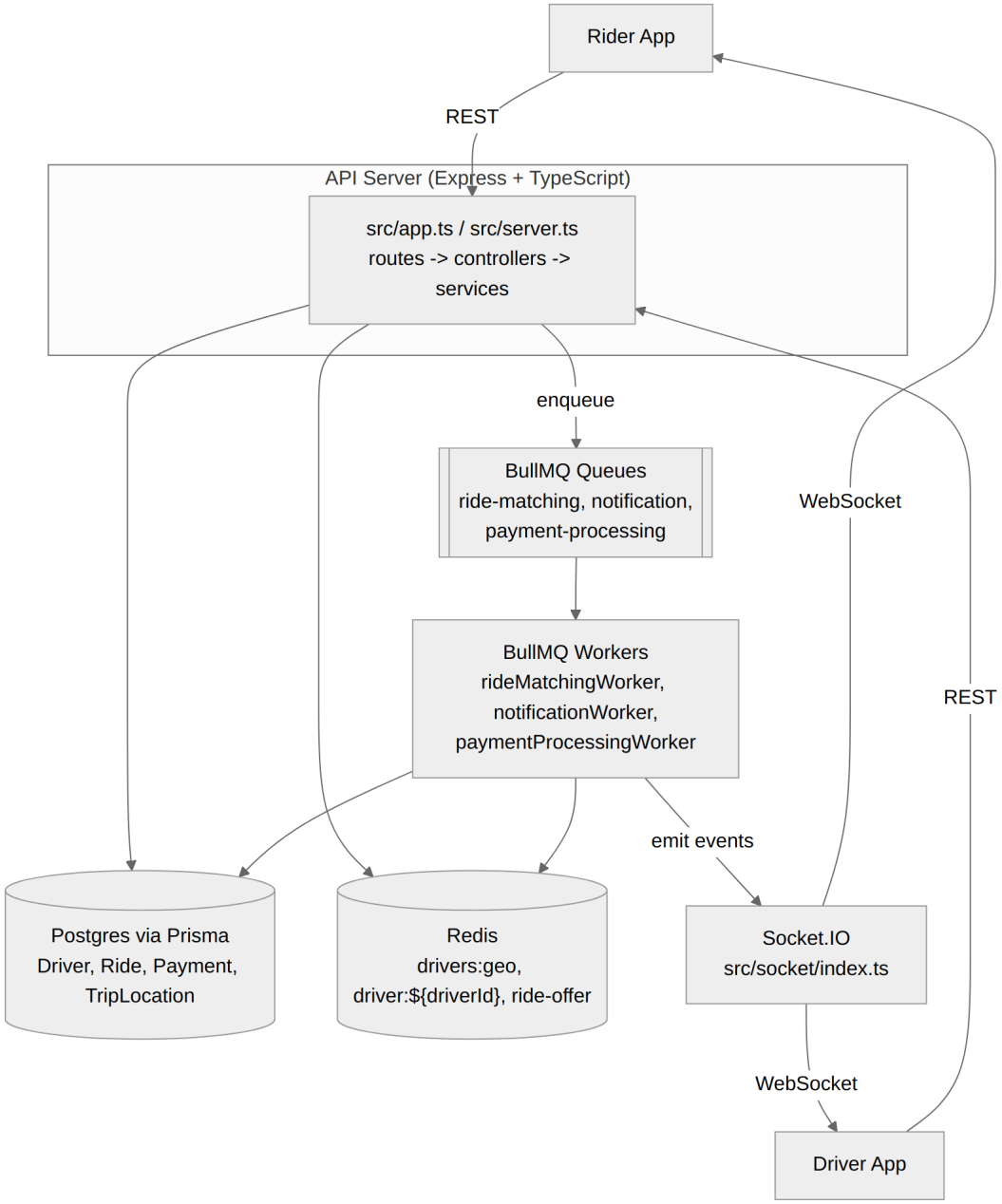
The Rider App calls `POST /v1/rides`; the API persists the `Ride` and enqueues a `ride-matching` job. `rideMatchingWorker` queries `drivers:geo` in Redis for nearby drivers, checks each candidate's live status in `driver:${driverId}`, and pushes a `ride-offer` (carrying a `bidToken`) to the chosen driver's Socket.IO room. The Driver App calls `POST /v1/drivers/:id/accept`; the API verifies the `bidToken`, runs a Prisma transaction to assign the ride and reserve the driver, and broadcasts `ride-assigned` to the ride's room. Trip progress (`STARTED`, `COMPLETED`) updates `Ride.status` in Postgres and streams `location-updates` to the rider. On completion, a `Payment` is created and processed asynchronously by `paymentProcessingWorker`, and `notificationWorker` fans out events such as `payment-successful` through Socket.IO.

Scalability Highlights (see full notes at the end)

- Stateless API server → scales horizontally behind a load balancer.
- Redis Geo (`drivers:geo`) → near-constant-time nearest-driver lookup instead of scanning Postgres.

- BullMQ workers → matching, notification, and payment work scale independently of the API tier.
- Socket.IO rooms → events are scoped to `driver:${driverId}` / `ride:${rideId}`, not broadcast globally.
- Postgres → remains the durable system of record while absorbing none of the per-second location traffic.

HLD Mermaid Diagram



LLD Explanation

Request Path: Routes → Controllers → Services → DB/Redis/Queue

- `src/routes/drivers.ts` and `src/routes/rides.ts` declare endpoints and delegate to controllers.
- `src/controllers/driversController.ts` and `src/controllers/ridesController.ts` validate input and shape responses only — no direct DB/Redis access.
- `src/services/driversService.ts` and `src/services/ridesService.ts` hold all business logic: Prisma reads/writes, Redis reads/writes (`src/redis/geo.ts`, `src/redis/rideOffers.ts`), and BullMQ enqueueing.

Driver Authentication & Online Flow

1. Driver signs up / logs in through `src/routes/drivers.ts` → `driversController` → `driversService`, which creates/looks up the `Driver` row via Prisma.
2. To go online, the driver app calls a driver-status endpoint; `driversService`:
   - Sets `Driver.status = AVAILABLE` in Postgres.
   - Writes `driver:${driverId}` in Redis with status `AVAILABLE`.
   - Adds the driver's position into `drivers:geo` via `src/redis/geo.ts`.

Driver Location Update (every 1s)

1. `PATCH /v1/drivers/:id/location` → `driversController.updateLocation` → `driversService.updateLocation`.
2. The service always refreshes the driver's position in `drivers:geo` (Redis), so matching stays accurate.
3. **Only if the driver has an active ride** does the service also insert a `TripLocation` row (`driverId`, `rideId`, `latitude`, `longitude`) via Prisma — this keeps a per-ride trail without writing every idle driver's position to Postgres every second.
4. If on an active ride, the service emits `location-updates` via Socket.IO to room `ride:${rideId}`.

Ride Creation & Matching Flow

1. `POST /v1/rides` → `ridesController.createRide` → `ridesService.createRide`.
2. The service creates the `Ride` row with `status: RideStatus.MATCHING` and enqueues a job on the `ride-matching` queue with the ride ID and pickup coordinates.
3. `rideMatchingWorker` consumes the job:
   - Queries `drivers:geo` (Redis) for the nearest candidate driver IDs.
   - For each candidate, checks `driver:${driverId}` in Redis and proceeds **only if status is `AVAILABLE`**.

    - Generates a `bidToken` (signed using `JWT_BID_TOKEN`) carrying `rideId` and `driverId`.
    - Stores the offer in Redis via `src/redis/rideOffers.ts` (keyed by ride/driver, TTL-bound) so it can later be verified and cannot be replayed.
    - Emits `ride-offer` over Socket.IO to room `driver:${driverId}`.
4. If the offer window expires with no acceptance, the worker retries the next nearest candidate or marks the ride `EXPIRED` once candidates are exhausted.

Bid Token Generation & Verification

- **Generated** by `rideMatchingWorker` as a JWT signed with `JWT_BID_TOKEN`, embedding `rideId` + `driverId` + expiry.
- **Sent** inside the `ride-offer` Socket.IO event to `driver:${driverId}`.
- **Received** back from the Driver App on `POST /v1/drivers/:id/accept`.
- **Verified** by `driversService.acceptRide`: validates the JWT signature/expiry, then cross-checks the stored offer in `src/redis/rideOffers.ts` to confirm this driver still holds an active, unconsumed offer for this ride.

Driver Accept, Pickup, and End-Ride Flow

1. On a verified `bidToken`, `ridesService` runs a **Prisma transaction**:
    - `Ride.status = ASSIGNED`, `Ride.driverId = driverId`.
    - `Driver.status = RESERVED`.
2. After commit, `driversService` sets `driver:${driverId} = RESERVED` in Redis and clears the consumed offer in `rideOffers.ts`.
3. `notificationWorker` (via the `notification` queue) emits `ride-assigned` to room `ride:${rideId}`.
4. **Pickup:** ride moves to `Ride.status = STARTED`; `driver:${driverId}` moves to `BUSY`.
5. **End ride:** ride moves to `Ride.status = COMPLETED`; `driver:${driverId}` moves back to `AVAILABLE`; `Ride.fareCents` is set.

Payment Flow

1. On ride completion, `ridesService` creates a `Payment` row (`rideId`, `amount = fareCents`, `status: PaymentStatus.PENDING`, `provider: "mock-psp"`) and enqueues a job on `payment-processing`.
2. `paymentProcessingWorker` calls the provider and updates `Payment.status` to `SUCCESSFUL` or `FAILED`.
3. On success, a `notification` job carrying `payment-successful` is enqueued for `notificationWorker`.

Notification Flow

- `notificationWorker` consumes the `notification` queue and resolves the target Socket.IO room (`driver:${driverId}` or `ride:${rideId}`) per event (`ride-assigned`, `payment-successful`).

- `src/socket/index.ts` authenticates each socket on connection and joins it to its relevant rooms; emits are always room-scoped, never global.

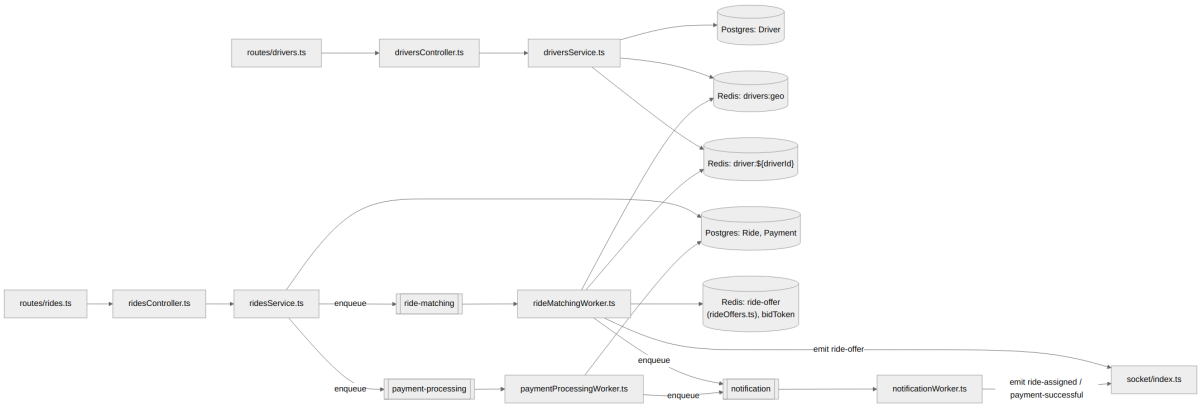
Redis State Transitions — `driver:${driverId}`

`OFFLINE → AVAILABLE → RESERVED → BUSY → AVAILABLE` (loop), with `RESERVED → AVAILABLE` possible if a ride is cancelled/expired before pickup, and `AVAILABLE → OFFLINE` when the driver logs off.

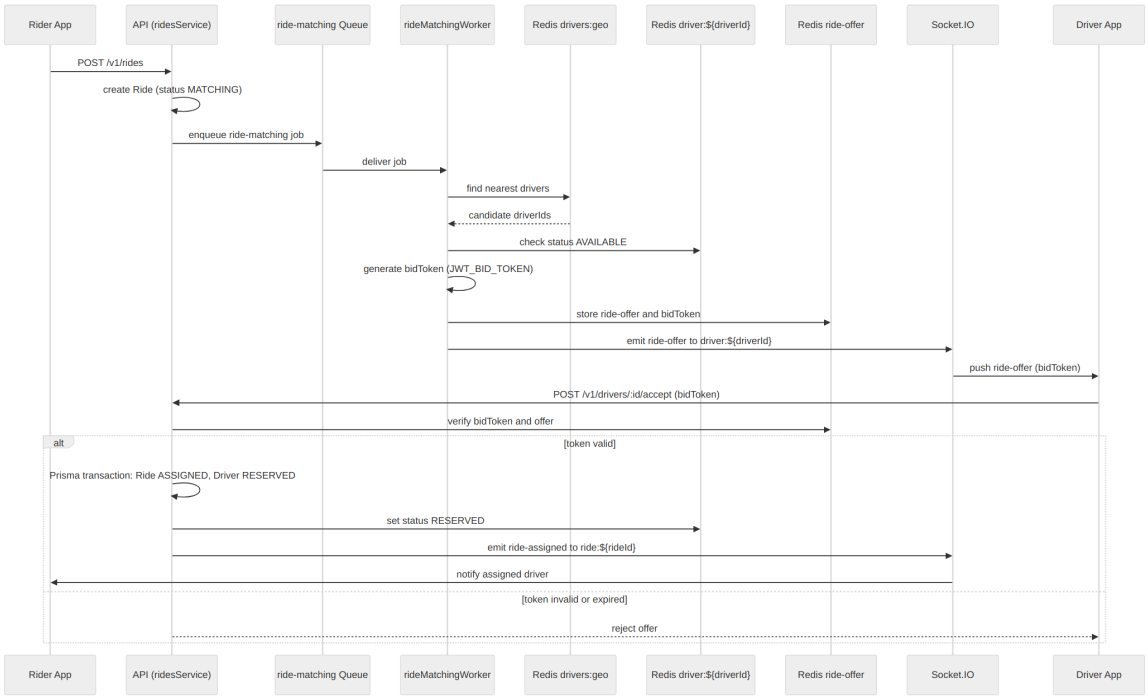
Prisma State Transitions

- **RideStatus:** `REQUESTED → MATCHING → ASSIGNED → STARTED → COMPLETED`, with `CANCELLED` reachable before `STARTED` and `EXPIRED` reachable from `MATCHING` if no driver accepts.
- **DriverStatus:** mirrors the Redis transitions above, kept in sync in Postgres at each step.
- **PaymentStatus:** `PENDING → SUCCESSFUL` or `PENDING → FAILED`, set by `payment-ProcessingWorker`.

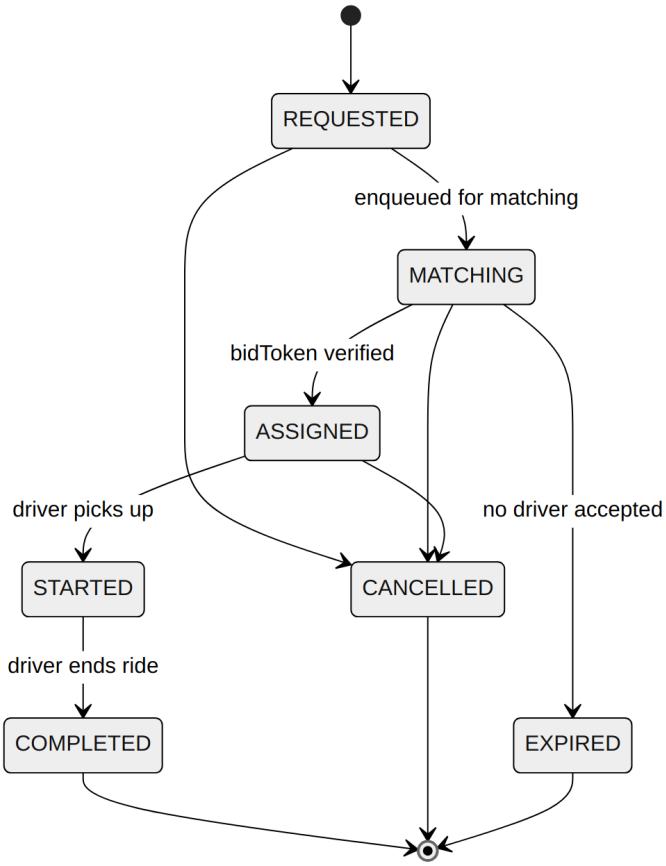
LLD Mermaid Diagram



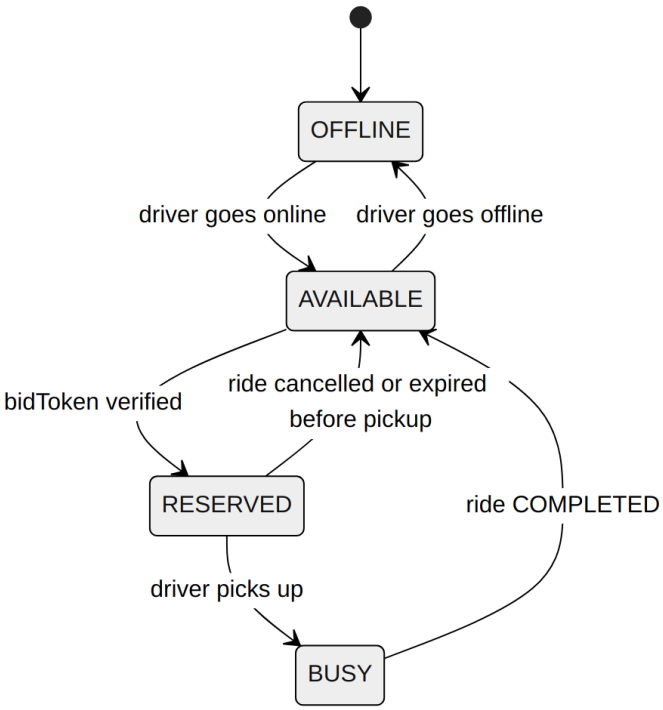
Booking Sequence Diagram



RideStatus State Diagram



DriverStatus State Diagram



Scalability Notes

- **Stateless API server** — no in-memory ride/driver state in `src/app.ts`; any instance can serve any request, so the API tier scales horizontally behind a load balancer.
- **Redis Geo (`drivers:geo`) for nearby-driver lookup** — `rideMatchingWorker` queries Redis instead of scanning `Driver`/`TripLocation` rows in Postgres, keeping matching latency low regardless of fleet size.
- **BullMQ workers for async processing** — `rideMatchingWorker`, `notificationWorker`, and `paymentProcessingWorker` each consume their own queue and can be scaled (more worker processes) independently of the API and of each other, based on load.
- **Socket.IO rooms for targeted events** — `driver:${driverId}` and `ride:${rideId}` scope every emit to the clients that actually need it, avoiding global broadcast overhead; for multi-instance deployments, Socket.IO's standard Redis adapter lets all instances share room membership without extra infrastructure.
- **Postgres as durable source of truth, shielded from high-frequency writes** — `Driver`, `Ride`, and `Payment` stay authoritative, while the 1-second location stream is absorbed by Redis; `TripLocation` is written only when a ride is active, keeping Postgres write volume proportional to active rides, not to the whole driver fleet.
- **Redis ride-offer TTL + bidToken** — prevents double-acceptance races without